ENGINEERING HISTORY

VirtualGlove Engineering Journey

One week of hypotheses, measurements, experiments, and play tests.

2026 EDITION / IAIN BENNETT / MIT LICENSE

This document records how VirtualGlove was developed and validated. It is the project's technical journey: the hypotheses tried, measurements gathered, approaches retired, and lessons that made the system playable. For the system as it exists today, read Architecture and flows. For exact performance evidence, read Native movement response and validation. To reproduce supported measurements, use the version-matched Engineering Toolkit.

The short version

In one intensive week, a camera-based hand tracker became a two-device game controller with shared gesture recognition, automatic game selection, native Power Glove emulation, a family-friendly learning system, secure network transport, recovery tooling, installers, Help, and repeatable validation.

The process worked because each uncertain part was separated and tested:

1. Establish a dependable joystick path before attempting native emulation.
2. Treat protocol notes as hypotheses and confirm packet behaviour against the exact ROM, emulator traces, and visible game response.
3. Measure camera capture, inference, transmission, receiver publication, core consumption, and display response as separate stages.
4. Keep experimental choices reversible until live play proved which felt best.
5. Use saved clips and headless tests for repeatability, then use human play tests for the final judgement that instruments cannot supply.
6. Preserve a working fallback whenever a deeper experiment was incomplete.

Milestones and decisions

- **Shared recognition first.** One calibration and gesture layer was made to serve every profile. FCEUmm remained the reliable joystick-output path.
- **Evidence-driven native input.** A separately named `lr-nestopia-powerglove` core was built without changing stock Nestopia. The supplied Super Glove Ball ROM, controlled traces, existing emulator code, manuals, and NESdev notes were used in that order to validate behaviour.
- **Continuous X/Y experiments.** Fixed smoothing, bounded speed response, direct latest-coordinate output, and optical flow were compared. Latest coordinate won live testing because it stayed attached to the visible hand; bounded response and optical flow remain historical evidence rather than user-facing gameplay choices.
- **Recognition became the latency target.** Transport and receiver work were reduced to roughly millisecond-scale stages. Camera delivery, MediaPipe inference, and detector reacquisition were the meaningful remaining costs.
- **CPU before exotic acceleration.** MediaPipe 0.10.35 with four XNNPACK CPU threads outperformed the earlier runtime. MediaPipe GPU, MNN over Turnip/Vulkan, and ncnn sidecar experiments did not beat the complete CPU graph under the project's correctness and stability gates.
- **Capture was made tunable.** OpenCV stayed the compatible default. Direct V4L2, camera rate, exposure, gain, and one- or two-buffer delivery became capability-detected comparisons, supported by a camera-settings wizard.
- **Fast sweeps were treated as reacquisition problems.** A direction-aware search region, newer-frame selection, a short last-reliable-position hold, and quick first-fresh-coordinate recovery reduced backward jumps without adding prediction or overshoot.
- **Reliability became part of responsiveness.** Signed latest-only transport, emulator-aware native/joystick switching, standard camera recovery, and capability-gated USB port power cycling prevented stale state and shortened recovery from faults.

What the team learned

Small embedded savings add up, but only after the largest cost is identified. Removing a millisecond of preparation work matters; replacing a reliable graph with a fragile accelerator that saves nothing does not. A good benchmark must also preserve the behaviour people notice: continuity, correct direction, quick stops, fast reacquisition, and a hand that appears attached to the on-screen glove.

The most productive loop was: form one testable hypothesis, add measurement, run the same input through old and new lanes, keep the safer result, and then play the game. Configuration switches were temporary scientific controls, not permanent user complexity. Once evidence converged, the winning path became the default and the user interface became simpler.

The remainder of this document preserves the detailed movement, runtime, GPU, and reacquisition investigations that support those decisions.

Final CPU-pipeline boundary - 10 September 2026

The final refinement pass tested whether camera delivery or Linux scheduling was making palm reacquisition appear expensive. A sustained output-paused trace measured driver dequeue, MJPEG decode, inference pickup, graph execution, post-graph recognition, and task run-queue wait separately. The matched two-buffer direct V4L2 diagnostic sustained approximately 30 dequeues per second: camera dequeue cadence was 33.29/35.84 ms p50/p95, decode was 5.45/9.87 ms, and decode-to-inference pickup was 18.18/33.27 ms. Post-graph work was only 2.93/3.13 ms.

Ordinary landmark continuation measured 36.86 ms median. The 60 palm-detector frames measured 104.94/112.89 ms p50/p95 and skipped two application-visible camera frames at median while the graph was busy. Inference accumulated only 132 ms of run-queue wait across the entire 60-second trace, so scheduling did not explain the detector cost. The delay began inside MediaPipe's recovery path.

A separate replay lowered landmark-tracking confidence to 0.10 and 0.20 in an attempt to avoid detector entry. Both produced the same eight fast-sweep misses as the 0.35 control and increased reacquisition p95 from about 102 ms to 126-128 ms. The broader recovery clip likewise found no worthwhile trade-off. The accepted defaults therefore remain tracking confidence 0.35, palm-detection confidence 0.45, four CPU threads, the 2.25 direction-aware search region, and Latest-coordinate output. Further improvement would require a different detector architecture and must earn its place through isolated testing; it is not part of the 0.4.0 production path.

Historical bounded speed-sensitive replacement - 7 September 2026

The former bounded native motion experiment used completed MediaPipe palm observations and separates resting noise from intentional velocity. It measures consecutive MediaPipe coordinates using capture timestamps and normalizes velocity by the player's directional reach. Calibration noise creates an elliptical X/Y resting region with hysteresis. Above it, one coherent two-dimensional newest-coordinate weight rises smoothly from `0.70` to `1.00` at 1.50 calibrated reach spans per second. Large travel and meaningful reversals are therefore direct. A stop settles inside the measured noise region on its first fresh result and exactly on the next, avoiding independent per-axis snaps.

The runtime cap is unconditionally `1.00`. Historical `motion_coordinate_boost` and `motion_coordinate_max` fields remain loadable, but cannot create extrapolation. No moving-average window, queue, replay, prediction, or core-side filter was added.

MediaPipe is the only live coordinate source because optical-flow movement was reported as jerky and unreliable even though the underlying MediaPipe tracking remained usable. The optical-flow implementation is retained as inactive source and historical trace support, but `motion_tracking` no longer enables it. The Dashboard no longer exposes the bounded curve; live movement uses direct Latest coordinates.

The initial MediaPipe-first deployment exposed two calibration mistakes in the bounded curve. Its `0.70` slow-follow weight was referenced to 60 Hz even though the Controller measured roughly 9-10 completed inferences per second; temporal normalization therefore raised most real samples to nearly one-to-one. The saved test calibration also contained saturated `1.0` jitter values, which the curve interpreted as a large image-space dead zone. Follow weighting now uses a 100 ms reference interval, and saturated jitter falls back to the small fixed floor while preserving the saved center, scale, and wrist.

`benchmark-vision-replay.py` version 2 retains timestamps and cue definitions in its aggregate local report. `benchmark-native-motion-curve.py` consumes that report, compares the former `.70 / 15 / 1.30` experiment, a capped error-driven reference, the actual speed curve, and direct latest coordinates, then evaluates 27 bounded candidates. It reports neutral span, selected-to-filtered error, medium 90% response, large first-sample misses, reversals, overshoot, continuity, and recognition age when present. The temporary physical clip is not currently available on the development Mac, so recorded-clip and synchronized live camera-to-display validation remain pending.

The 0.4.0 production choice is Latest coordinate. It publishes every valid reach-clamped MediaPipe point directly during continuous tracking. After a brief dropout, one result that contradicts established motion-or is unusually distant without

strong forward alignment-is held until the next fresh result. Live tracing showed that allowing strongly aligned forward recovery immediately avoided the earlier compound catch-up jump while inserting no unnecessary holds in the tested gameplay window. This is a reacquisition guard, not smoothing or prediction.

The runtime now rejects malformed/non-finite landmark geometry, retains the calibration-compatible five-point wrist/knuckle average as the production anchor, and exports three alternative anchors for comparison. It clamps the selected point to player reach before processing. The historical comparison applied the same clamp to both Latest and Bounded lanes. Runtime clears history on stale/lost input and uses the frame capture timestamp rather than inference-start time for freshness.

The gameplay and dot recordings contain aggregate timings and sampled validity; they do not retain recognized, flow, selected, and filtered coordinates for each individual move. They cannot establish how many camera images or presented game frames contained intermediate hand positions. Receiver publications are not necessarily new measurements or different coordinates.

Observed evidence

30-second window	Valid Controller samples	Observed tracking losses
Earlier flow revision, dot movement	579/591 (98.0%)	6
Recognition fallback, fast dot movement	447/589 (75.9%)	34
Recognition fallback, actual gameplay	568/590 (96.3%)	3

A later MediaPipe-only, Dashboard-closed status run measured 96.5% detected samples, 65.4 ms median and 76.2 ms p95 source age, 52.2 ms median and 58.3 ms p95 inference, 2 ms p95 send work, and about 16 distinct native updates per second. This is a software-stage sample, not physical hand-to-display latency.

These were different physical movements, not controlled before/after trials. In gameplay, 395/590 polls reported `flow_unavailable`, 61 reported `flow_seed_unavailable`, and one reported `correction_budget`. Fallback therefore frequently delivered recognition coordinates rather than a newer flow estimate. These are sampled frame counts, not distinct recognition-result counts. The receiver observed 1,367 distinct valid publications in 30 seconds, not 1,367 new recognized positions or displayed images.

Recognition runtime and Adreno 702 research - 9 September 2026

What the MediaPipe graph means

The MediaPipe graph is the complete hand-processing pipeline, not one neural network. It connects frame preparation, palm detection, hand-region geometry, the landmark model, tracking between detections, and result delivery. The palm and landmark neural networks are nodes inside that graph. During continuous tracking, the landmark model uses the previous hand region. When that evidence fails, the graph runs the more expensive palm detector to reacquire the hand.

VirtualGlove consumes the graph's 21 landmarks once per fresh result. The five-point palm anchor and gesture calculations happen after MediaPipe has already calculated all landmarks; reducing the number of points averaged by VirtualGlove would therefore not reduce neural-network work.

MediaPipe 0.10.35 CPU promotion

The release now ships only the ARM64/Python 3.12 MediaPipe 0.10.35 wheel. The production lane remains the complete MediaPipe Hands graph with four XNNPACK CPU threads, fused full-colour preparation, a 0.35 tracking-confidence threshold, the 2.25 hand-search region, and Latest-coordinate native X/Y. Gesture rules, calibration, reach, mappings, and controller transport did not change.

The same retained 736-frame fast-sweep clip was replayed through the former 0.10.18 runtime and the selected 0.10.35 runtime with identical settings:

Runtime	Inference p50 / p95	Detection continuity	One-frame misses
Historical 0.10.18	36.12 / 60.18 ms	97.96%	15
Shipped 0.10.35	34.03 / 46.60 ms	97.96%	15

Landmark-continuation p95 fell from 51.10 ms to 44.29 ms. Palm-reacquisition p95 fell from 149.48 ms to 120.16 ms. A separate live Super Glove Ball trace improved capture-to-send p95 from 126.84 ms to 91.16 ms and delivered all 3,408 observed samples to RetroPie with no trace drops. The player reported that both ordinary tracking and return-to-space reacquisition felt zippier.

An output-paused ten-minute soak exercised the complete camera and graph. It completed without a crash, camera loss, worker error, or thermal throttling. The hottest reported thermal zone reached 61.7 degrees Celsius. Container memory rose during initialization, then remained at an approximately 484 MiB plateau. The blank-room portion deliberately stressed repeated palm detection; the matched clip supplies the continuity evidence that an unattended room cannot.

The packaged wheel was rebuilt from upstream MediaPipe commit `f8ef212d5c962c0e853db7e59d217056b187084b`. Its release metadata removes unused JAX and JAXLIB dependencies and pins headless OpenCV 4.11.0.86. The rebuilt integrity record has SHA-256 `3f09815d9f6c41d828cd71c9ba477c24a63850908876dfc8b095a882c790e562`. An isolated ARM64 import confirmed MediaPipe `0.10.35+powerglove.cpu1`, OpenCV `4.11.0`, and no installed JAX module.

Why production still uses the CPU

The UNO Q exposes a genuine Adreno 702 at 845 MHz through Mesa's Turnip driver. Tests reached the hardware through EGL/OpenGL ES, OpenCL through Rusticl, and Vulkan. Hardware access was therefore confirmed; the limitation was the tested inference software path, not an absent GPU.

Exact or representative lane	CPU result	Adreno result	Decision
MediaPipe 0.10.35 hand landmark continuation	32.74 ms p50	384.12 ms p50	GPU rejected
MNN 3.6.1 landmark-lite model	19.7 / 26.8 ms	43.9 / 44.0 ms	Vulkan slower and numerically incompatible
MNN 3.6.1 palm-lite model	36.3 / 44.3 ms	77.9 / 78.2 ms	Vulkan slower and numerically incompatible
ncnn exact landmark model	25.0 / 38.7 ms	385.5 / 386.9 ms	Vulkan slower and numerically incompatible

The MediaPipe OpenGL graph repeatedly synchronized small tensors and was more than ten times slower than XNNPACK. The experimental Linux OpenCL port reached real FD702 graph construction but failed while creating an

image from a buffer. MNN's per-layer profiler showed time distributed across convolution and depthwise-convolution work rather than one fixable synchronization tail.

An ncnn CPU sidecar did reproduce the exact landmark model and closely matched MediaPipe coordinates, but its complete replay measured 36.1-36.5 ms p50 and 45.7-52.3 ms p95 with 98.68% continuity. Image preparation, process handoff, and duplicated tracking geometry removed its model-only advantage. It remains a repository research tool rather than an installed backend.

Qualcomm QNN remains a possible future lane only if a redistributable UNO Q runtime becomes available and proves end-to-end improvement. No confidence, reach, or gesture tuning can repair hundreds of milliseconds spent inside a delegate. Any future accelerator must beat the current camera-to-coordinate p95 by at least 20%, preserve ordered newest-sample delivery and recognition within one percentage point, add no jitter or false gestures, and remain thermally stable for ten minutes.

The supporting upstream references are MediaPipe's framework documentation, hand-tracking graphs and model documentation; Qualcomm's AI Engine Direct and TensorFlow Lite delegate documentation; MNN's Vulkan backend; ncnn's Vulkan documentation; and Mesa's Freedreno/Turnip device support. Exact commits, converted-model hashes, profiler outputs, and temporary binaries remain engineering evidence and are deliberately excluded from ordinary installers.

Historical smoothing-only experiment

Before the speed-curve replacement, `scripts/analyze-motion-samples.py` exercised the error-driven native movement engine with an ideal instantaneous measured-position step. It assumed 60 updates/second, no input noise or loss, and the former code defaults (minimum .70, maximum 1.00, motion boost 4.00). The following table is retained as historical evidence for that implementation.

Step in normalized camera X	Time from first step sample to 95% of target
0.010	200 ms
0.025	200 ms
0.050	167 ms
0.100	Immediate on first sample
0.200	Immediate on first sample

With smoothing disabled, every modeled step passes through on its first sample. That comparison does **not** measure the jitter cost of disabling smoothing. The elapsed times exclude capture, recognition, transport, game simulation and display. They must not be added to independent timing percentiles. The final five-percent criterion includes a small settling tail; it is not a delay before movement begins.

The speed-dependent behavior explained the problem: the former coefficient used position error, then adjusted for elapsed time. It was adaptive, but was not a One Euro velocity-based filter. Large errors can bypass damping while small aiming corrections retain it. This is a plausible contributor to the reported difficulty catching and directing the ball, not proof of the complete cause.

The script now exercises the bounded speed curve and labels its parameters in new reports. Prediction accuracy cannot be tested from these aggregate reports. A predictor can extrapolate an old measurement, but abrupt stops and reversals can cause overshoot. Do not fit or enable prediction using these reports as ground truth.

Prepared trace extension

The optional finite diagnostic trace now includes, on each vision event:

- New recognition completion, its original capture timestamp, X/Y, confidence, and detection flag. In-flight frames contain no fabricated recognition event.
- Accepted anchor's original capture timestamp.
- Attempted optical-flow X/Y and a separate acceptance flag; a correction that exceeded its budget must not be counted as accepted flow.
- Selected normalized X/Y, fallback reason, and invalid-input reason.
- Final filtered normalized X/Y, smoothing parameters, and mapped native axes.

Capture sequence/timestamp and transport sequence remain available for correlation. Raw and filtered normalized coordinates share camera space; native axes use the player's reach mapping. Invalid selected coordinates are null. A repeated source is identifiable by anchor timestamp and must not be treated as a fresh velocity measurement. The existing trace remains opt-in, finite, buffered, and asynchronous. No video is recorded by this extension.

The instrumentation was subsequently deployed with the medium-jump trial below; trace collection has not yet been enabled. Before the next physical trial, enable a finite trace, then record the real hand and cabinet screen together with the iPhone. Test small corrections, a large move, a stop, and a reversal. Trace replay can compare smoothing outputs, while video supplies physical timing and overshoot evidence. Cross-device clocks require alignment; do not subtract them directly.

Reproduce the offline report with:

```
SH
python3 scripts/analyze-motion-samples.py \
  --samples-dir /tmp/uno-dot-baseline-x8heur1f \
  --output /tmp/new-motion-analysis.json
```

Analyze one finite per-frame trace without replaying controller input:

```
SH
python3 scripts/analyze-motion-trace.py /tmp/controller.trace.json \
  --output /tmp/controller-motion-analysis.json
```

For a controlled directory whose filenames follow `min0.70-boost8.trace.json`, compare every configuration with:

```
SH
python3 scripts/compare-motion-matrix.py /tmp/controller-motion-matrix \
  --output /tmp/controller-motion-matrix.json
```

Both tools create new reports rather than overwriting existing evidence. Their settling estimates describe normalized software coordinates, not physical hand-to-screen latency.

Medium-jump and bounded extrapolation trial

This historical experiment let `motion_coordinate_boost` override the boost in `update_native_motion`; null preserved the baseline value. The Controller trial used boost 15.0 instead of the baseline 4.0, with minimum .70. An additional experimental `motion_coordinate_max` cap of 1.30 allowed bounded extrapolation beyond the newest measured position. This was an error relative to the filtered position, not a speed threshold or a percentage of the game screen. It does not remove recognition age.

The actual-engine 60 Hz step model gives 0 ms additional settling time for .04 and .05 steps with boost 8, compared with 183 and 167 ms respectively at boost 4. A .005 step remains damped (200 ms to 95%, previously 217 ms). Noise and physical response still require the paired recording. Baseline synchronous tracking was unchanged. The bounded speed-sensitive replacement now ignores these legacy overrides during native movement while continuing to load them. Trace collection must be explicitly enabled for a physical test.

Six-configuration trace matrix

The matrix varied minimum smoothing and experimental motion boost while holding the 250 ms freshness limit, 320 px flow width, 25 ms correction budget, and maximum smoothing 1.00 fixed. Each window produced a finite trace with zero dropped records. The analysis uses `scripts/compare-motion-matrix.py`.

Minimum / boost	Vision events	Valid %	Losses	Age p95 (ms)	Medium settling p95 (ms)
0.70 / 4	834	69.18	14	104.5	0.0
0.70 / 8	751	56.72	17	175.0	0.0
0.70 / 12	895	90.17	7	87.2	84.6
0.85 / 4	884	89.37	4	104.1	114.0
0.85 / 8	886	97.74	1	85.4	96.9
0.85 / 12	897	97.44	2	83.1	63.8

These windows did not contain a controlled, repeated movement sequence, so the rows cannot select a production candidate. Validity and recognition age varied enough to confound settling comparisons. The results are a harness check and preliminary range, not proof that 0.85 / 12 is better. The live setting was restored to 0.70 / 8; calibration was unchanged.

Controlled three-window follow-up

The follow-up repeated the same center, small-correction, medium-jump, reversal, and center sequence for the baseline and two candidates. All three finite traces had zero dropped records.

Configuration	Vision events	Valid %	Losses	Age p95 (ms)	X error median / p95	Medium settling p95 (ms)
0.70 / 8 (baseline)	778	94.47	6	87.5	0.0011 / 0.0122	120.8
0.85 / 8	883	92.87	5	91.5	0.0006 / 0.0052	17.9

Configuration	Vision events	Valid %	Losses	Age p95 (ms)	X error median / p95	Medium settling p95 (ms)
0.85 / 12	894	90.27	7	111.5	0.0004 / 0.0041	112.1

The controlled follow-up showed `0.85 / 8` was a useful intermediate smoothing candidate, but later gameplay testing found it too floaty. The live experiment was returned to minimum smoothing 0.70 and then raised to boost 15 with cap 1.30. Those later values are a user-driven exploratory setting, not a production default; repeat controlled traces before promoting them.